
模板

复习

- | | |
|----------------|----|
| 1. 异常处理概述 | 了解 |
| 2. 标准异常 | 熟悉 |
| 3. try/catch语句 | 重点 |
| 4. 抛出异常 | 重点 |
| 5. 捕获异常 | 重点 |
| 6. 自定义异常 | 重点 |
| 7. 嵌套类和局部类 | 熟悉 |

目录

- | | |
|-------------|----|
| 1. 模板概述 | 了解 |
| 2. 函数模板 | 重点 |
| 3. 函数模板的原理 | 熟悉 |
| 4. 函数模板的实例化 | 重点 |
| 5. 类模板 | 重点 |
| 6. 类模板实例化 | 重点 |
| 7. 模板别名 | 重点 |

模板概述

有时候在设计程序的时候会遇到这样一种情况：需要设计的几个类，其功能都是一样的，仅仅只是需要操作的数据类型不同。例如需要创建一个数组类，该数组可能是 int 整型数组，也可能是 double 类型数组、string 类型数组等。解决类似的问题，固然可以将所有的类都设计一遍，但是在 C++ 中，有更好的方法，就是设计一个**模板类**。

C++ 提供了多种代码重用机制，在前面介绍派生和继承时，派生类可以继承基类中的成员变量和成员函数。

模板是另一种代码重用机制。

模板是一个非常强大的C++功能，STL的各种组件也是基于模板的。所以，无论是写程序了，还是读程序，都有必要了解一下C++的模板。

总之，**模板可以提高代码的可用性, 减少开发的代码量和工作量.**

函数模板

模板(Templates)使得我们可以生成通用的函数，这些函数能够接受任意数据类型的参数，可返回任意类型的值，而不需要对所有可能的数据类型进行函数重载。这在一定程度上实现了宏（macro）的作用。它们的原型定义可以是下面两种中的任何一个：

```
template <class identifier> function_declaration;
```

```
template <typename identifier> function_declaration;
```

上面两种原型定义的不同之处在关键字class 或 typename的使用。它们实际是完全等价的，因为两种表达的意思和执行都一模一样

函数模板

函数模板的格式：

//T代表的是一种类型

//也可以用class来代替typename

template<typename T1,typename T2,...,typename Tn>

返回类型 函数名(参数列表)

{

 //函数体

}

函数模板

想象一下，如果现在让你实现交换两个数的值（没说是整数还是小数），如何实现？

函数模板

C++的函数重载使得用于交换不同类型变量的函数可以拥有相同的函数名

// 交换两个整型

```
void Swap(int& x, int& y)
```

```
{
```

```
    int tmp = x;
```

```
    x = y;
```

```
    y = tmp;
```

```
}
```

// 交换两个双精度浮点型

```
void Swap(double& x, double& y)
```

```
{
```

```
    double tmp = x;
```

```
    x = y;
```

```
    y = tmp;
```

```
}
```

重载的多个函数仅仅只是类型不同，代码的复用率比较低，只要出现新的类型需要交换，就需要新增对应的重载函数。

代码的可维护性比较低。

如何改写成函数模板呢？

函数模板

//交换两个数的函数可以写成如下形式：

template<typename T>//或者template<class T>

void Swap(T& x, T& y)

{

 T tmp = x;

 x = y;

 y = tmp;

}

函数模板的原理

函数模板本身并不是函数，是编译器用使用方式产生特定具体类型函数的模具。所以其实模板就是将本来应该让程序员做的重复的事情交给了编译器。

在编译器编译阶段，对于函数模板的使用，编译器需要根据传入的实参类型来推演生成对应类型的函数以供调用，彼此的地址也是不同的。比如，当用int类型使用函数模板时，编译器通过对实参类型的推演，将T确定为int类型，然后产生一份专门处理int类型的代码，对于double类型也是如此。

函数模板的实例化

```
#include <iostream>
using namespace std;
template<class T>
T add(T& v1,T& v2) {
    return v1 + v2;
}
int main() {
```

```
    int n1 = 100, n2 = 200;
```

```
    int n3 = add(n1, n2);//编译器根据实参a1和a2推演出模板参数为int类型
```

```
    cout << n3 << endl;
```

```
    double d1 = 1.23, d2 = 2.34;
```

```
    double d3 = add(d1, d2);//编译器根据实参d1和d2推演出模板参数为double类型
```

```
    cout << d3 << endl;
```

```
    //double d4 = add(n1, d2);//错误！在模板中，编译器一般不会进行类型转换操作
```

```
    //cout << d4 << endl;
```

```
    return 0;
```

```
}
```

用不同类型的参数使用模板时，称为模板的实例化。模板实例化分为隐式实例化和显示实例化。

隐式实例化：让编译器根据实参推演模板参数的实际类型

函数模板的实例化

怎么解决呢？

```
template<class T , class H>  
T add(T& v1,H& v2) {  
    return v1 + v2;  
}
```

函数模板 案例

```
#include <iostream>
using namespace std;
template<typename T>
void change(T& x, T& y) {
    T temp;
    temp = x;
    x = y;
    y = temp;
}
int main() {
    int a = 100, b = 200;
    change(a, b);
    cout << a << " " << b << endl;
    double c = 1.112, d = 23.12;
    change(c, d);
    cout << c << " " << d << endl;
    return 0;
}
```

类模板

C++也可以定义类模板（ class templates ），使得一个类可以有基于通用类型的成员，而不需要在类生成的时候定义具体的数据类型。

类模板

类模板的定义格式：

```
template<class T1,class T2,...,class Tn>
```

```
class 类模板名
```

```
{
```

```
    //类内成员声明
```

```
};
```

类模板

定义的类可以用来存储两个任意类型的元素

```
template <class T>
class Pair {
    T values [2];
public:
    Pair (T first, T second) {
        values[0]=first;
        values[1]=second;
    }
};
```

类模板实例化

类模板实例化需要在类模板名字后面跟<>，然后将实例化的类型放在<>中。

比如：

```
Pari<int> myInt(1,2);
```

```
Pari<double> myFloat(1.1,2.2);
```

类模板 案例

```
#include <iostream>
using namespace std;
template<class T >
class Test
{
    //普通类：类名就是类型;
    //类模板: 类名不是类型，类型是Test<T>

public:
    Test() {}
    // 类里面声明，类外面定义
    void setData(const T& x);
    void showDate();

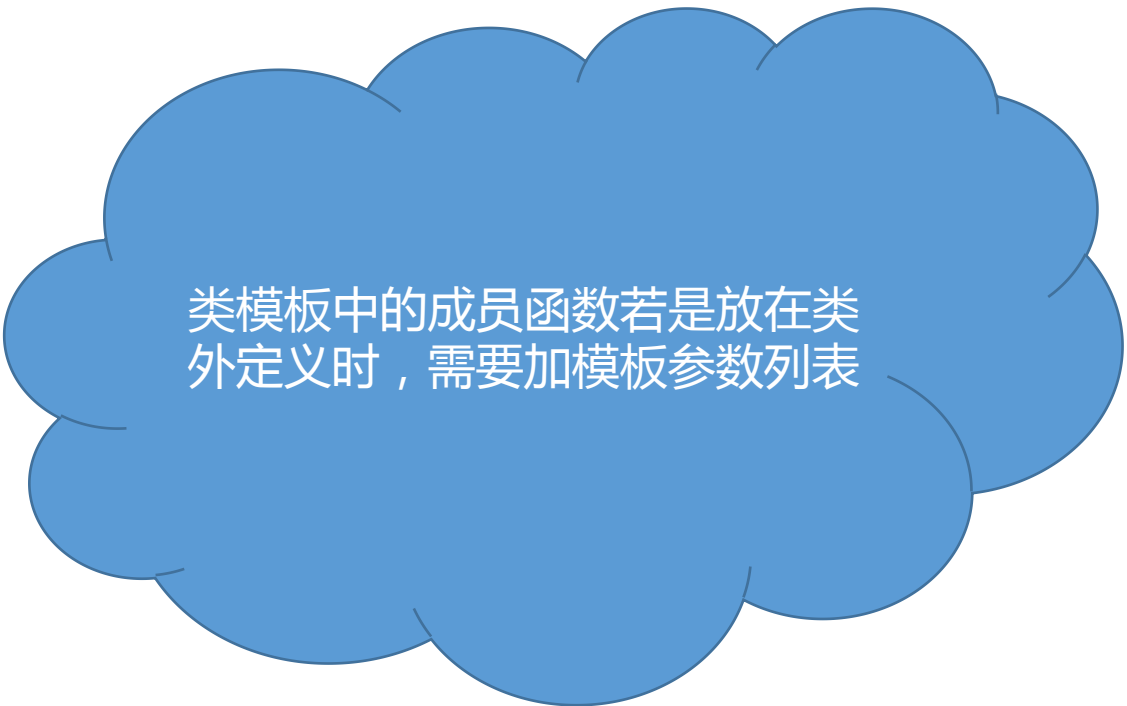
private:
    T num;
};

template<class T>
void Test<T>::setData(const T& x) {
    num = x;
}

template<class T>
void Test<T>::showDate() {
    cout << num << endl;
}

int main() {

    Test<int> t1;
    t1.setData(100);
    t1.showDate();
    return 0;
}
```



类模板中的成员函数若是放在类外定义时，需要加模板参数列表

模板别名

在写程序的过程中，总是希望写简短的代码，给一些复杂的名称简化，或者取一个简短的名字，于是就有了类型别名，用来重新定义复杂的名字。不管是模板还是非模板都需要类型别名，可以使用typedef为模板具体化指定别名。

```
typedef Test<int> t;

int main() {
    //Test<int> t1;
    t t1;
    t1.setData(100);
    t1.showDate();
    return 0;
}
```

模板别名

如果经常编写类似于上面typedef的代码，如果代码量过大的话，那么你可能会忘记能是什么意思，这样的话就不能叫做简化代码了，直接就是给自己找麻烦。所以

C++11新增了一项功能——使用模板提供一系列别名（**模板别名**），例如：

```
//typedef Test<int> t;
```

```
template <class T>
```

```
using t = Test<T>;
```

```
int main() {
```

```
    //Test<int> t1;
```

```
    //t t1;
```

```
    t<int> t1;
```

```
    t1.setData(100);
```

```
    t1.showDate();
```

```
    return 0;
```

```
}
```

练习 题目

编写一个函数模板，分别求一个组数里面数据的最大值，最小值和平均值。

练习 代码

```
#include<iostream>
using namespace std;

//求一个数列的最大值、最小值、均值
template<typename T>
void MathHelper(T num[], int size) {
    T max = num[0], min = num[0], total = 0;
    for (int i = 0; i < size; i++) {
        if (num[i] > max)
            max = num[i];
        if (num[i] < min)
            min = num[i];
        total += num[i];
    }
    //显示最大值、最小值、均值
    cout << "Max : " << max << endl;
    cout << "Min : " << min << endl;
    if (size == 0)
        cout << "Mean : " << 0 << endl;
    else
        cout << "Mean : " << total / double(size) << endl;
}

void main() {
    //测试
    int a[10] = { 2,1,3,6,5,8,9,7,11,22 };
    double b[10] = { 2.1,1.5,9.2,10.5,7.5,6.5,5.5,8.2,3.1,4.6 };
    MathHelper<int>(a, sizeof(a) / sizeof(a[0]));
    cout << endl;
    MathHelper<double>(b, sizeof(b) / sizeof(b[0]));
}
```

知识点总结

- | | |
|-------------|----|
| 1. 模板概述 | 了解 |
| 2. 函数模板 | 重点 |
| 3. 函数模板的原理 | 熟悉 |
| 4. 函数模板的实例化 | 重点 |
| 5. 类模板 | 重点 |
| 6. 类模板实例化 | 重点 |
| 7. 模板别名 | 重点 |

作业

1. 建立类模板input，在调用构造函数时，完成以下工作：

- (1) 提示用户输入；
- (2) 让用户输入数据；
- (3) 如果数据不在预定范围内，重新提示输入。

input型的对象应当按以下形式定义：

```
input ob ( "prompt message",min_value,max_value )
```

其中，prompt message是提示输入信息。可接受的最小值和最大值分别由min_value与max_value指定。

输入输出说明:

如果数据在给定的范围之内，则正确输出数据，否则提示重新输入

输入：

1 9 3

a z f

输出：

3

f

下节课预告

STL